

Soma: Achieving Low-Level Performance in a General-Purpose Dependently Typed Functional Language via Interaction Nets

Gabriel Di Lucca Minatel

Abstract

We present Soma, a dependently typed functional programming language that compiles interaction net reduction rules to native code via LLVM. Soma’s type system combines full dependent types with Quantitative Type Theory (QTT), which classifies every binding as erased (0), linear (1), or unrestricted (ω). This classification drives a three-tier compilation strategy: erased bindings generate no code, linear bindings compile to conventional LLVM IR with zero interaction net overhead and unrestricted bindings compile to type-specialized native interaction net reduction rules that preserve runtime fusion and enable automatic parallelism. The result occupies a previously empty point in the design space which is native performance comparable to Koka for the common (linear) case, with interaction net benefits (lazy sharing, fusion, strong confluence) for the unrestricted case, without the interpretation overhead of systems like HVM or Vine.

1. Introduction

Dependently typed functional languages occupy a well-explored point in the design space: expressive type systems paired with either interpretation or GC-backed native compilation. Lean compiles to C with reference counting, Idris 2 targets Scheme or JavaScript and Agda remains primarily an interactive proof assistant. These languages achieve correctness through their type systems but sacrifice performance characteristics that imperative languages take for granted: predictable allocation, cache-friendly data layout and the absence of garbage collection pauses.

At the other end, interaction net evaluators such as HVM [4], Vine [10] exploit the strong confluence property of Lafont’s interaction combinators [5, 6] to achieve automatic parallelism and optimal sharing. Every two non-interfering active pairs can reduce simultaneously without coordination, yielding extremely parallel execution by construction. However, these systems represent every value as a node in a heap-allocated graph, incurring interpretation overhead on every reduction step: tag dispatch, pointer chasing and queue management.

These two approaches appear to define a trade-off: native speed or interaction net semantics, but not both. We observe that no existing system occupies the upper-right quadrant of native speed *and* automatic parallelism via interaction nets.

Soma targets this quadrant through a key insight: rather than interpreting or compiling away the interaction net, we *compile the reduction itself* to native code. Most interaction rules (beta reduction, pattern matching, field projection, arithmetic) are statically predictable, the compiler knows at each call site exactly which rule applies. These compile to conventional LLVM IR with zero overhead. The remaining dynamic interactions (duplication commuting

through unknown values) are compiled to type-specialized native functions, preserving the graph topology that enables fusion and parallelism while eliminating interpretation overhead.

Quantitative Type Theory (QTT) [1, 7] makes this feasible. QTT annotates every binding with a quantity 0 (erased), 1 (linear), or ω (unrestricted) and the type checker enforces these annotations. The compiler trusts them absolutely:

- **Quantity 0:** The binding is a compile-time proof obligation where no code is generated.
- **Quantity 1:** The binding is consumed exactly once. The compiler emits direct LLVM IR identical to what a conventional compiled language would produce with no interaction net overhead.
- **Quantity ω :** The binding may be used multiple times. DUP and SUP nodes are generated and the interaction rules governing their behavior are compiled to native code per type. The graph structure is preserved for fusion and parallel reduction.

This paper presents the design and implementation of Soma’s memory management strategy, which achieves GC-free deterministic lifetimes through compiler-inserted DUP and ERA primitives derived from interaction net semantics.

2. Interaction Nets Overview

After linear logic was theorized, several researchers realized it could model functional computation with the bonus of fine-grained resource management. This led to variants of lambda calculus and the development of Interaction Nets by Yves Lafont in 1990.

Interaction nets are a form of graph rewriting system where computation is represented as the interaction between nodes (or agents) connected by edges. Each node represents a computational entity and the edges represent the flow of data between these entities.

2.1. Interaction Combinators

Interaction combinators are a minimal set of interaction net agents that can simulate any interaction net. They consist of three types of agents: the δ (duplicator) agent, the γ (constructor) agent and the ε (eraser) agent. These agents interact according to specific rules that define how they can be rewritten when they come into contact with each other.

These follow simple interaction rules: annihilation, commutation and erasure. Annihilation occurs when two agents of the same type collide, resulting in their removal from the net. Commutation happens when a constructor meets a duplicator. The duplicator clones the constructor and the constructor splits the duplicator. This is how copying propagates through a data structure. Finally, erasure happens when an eraser meets any agent, destroying it and its auxiliary ports.

The Interaction Combinators are Turing complete, meaning they can simulate any Turing machine. This property makes them a powerful tool for modeling computation in a way that is both efficient and expressive. The beauty lies in the locality and parallelism. Each interaction only involves two agents and their immediate connections. This means no global state, no shared memory, no synchronization needed. Any two independent interactions can happen simultaneously. This makes interaction combinators an ideal foundation for massively parallel computation.

2.2. Interaction Calculus

Interaction calculus is a higher-level language that maps onto interaction nets developed by HigherOrderCo. It is inspired by lambda calculus but adapted to the interaction net model. In interaction calculus, terms are represented as graphs and computation is performed through graph rewriting rules similar to those in interaction nets. Interaction calculus introduces constructs for defining functions, applying functions to arguments and managing resources in a way that aligns with the principles of interaction nets and linear logic.

3. Soma Language Design

Soma is inspired by Lean in the sense that it strives to have a minimal core language with universe polymorphism, dependent types and a powerful type system.

Due to this similarity and for convenience, we'll use Lean's syntax to illustrate some of the features of Soma, but the final syntax is bound to differ.

4. Memory Management

In this section, we formalize the memory management strategy employed by Soma. The system achieves deterministic, GC-free memory management through two primitives derived from interaction net semantics:

- **DUP** (duplication): produces two independently owned copies of a value.
- **ERA** (erasure): destroys a value and frees its resources.

These primitives are inserted automatically by the compiler based on static usage analysis. No programmer annotation is required.

4.1. The Duplication Problem

In a functional language without mutable state, every value is logically immutable. When a value is used in multiple places, the language must either share a single representation (as in garbage-collected languages) or produce independent copies (as in linear or affine systems). Soma takes the latter approach: every use of a value consumes it and multiple uses require explicit duplication.

Consider a simple example:

```
let x = expensive_computation in (f x, g x)
```

The variable x is used twice. The compiler transforms this into:

```
let x = expensive_computation in
let (x1, x2) = DUP(x) in
(f x1, g x2)
```

Each x_i is an independently owned copy consumed exactly once. The original x no longer exists after the DUP.

The problem becomes non-trivial in the presence of branching:

```
let x = expensive_computation in
case some_function x of
```

```

Left y => (f y, g x)
Right z => (h x, g x)

```

Here x is used once in the `Left` branch and twice in the `Right` branch. A naive approach that sums uses across branches overestimates the required copies. Before addressing lazy duplication (Section 4.4), we must first formalize the placement of DUP and ERA nodes.

4.2. Usage Counting and DUP Placement

The compiler must determine, for each binding, where to insert DUP and ERA nodes so that every value is consumed exactly once on every execution path. We consider three progressively refined strategies to motivate the final design.

4.2.1. Naive Approach: Additive Counting

The simplest approach sums all uses of a variable across all branches:

```

let x = expensive_thing in
if cond then
  work x x      -- 2 uses of x
else
  work2 x       -- 1 use of x

```

Additive counting yields 3 total uses, causing the compiler to emit 2 DUP nodes at the binding site of x , producing 3 copies. In the `else` branch, 2 of the 3 copies are unused and must be explicitly erased. This is correct but wasteful. The actual maximum simultaneous need is 2 (from the `then` branch).

4.2.2. Binding-Site Placement with Max-Counting

A better approach counts usage **per branch** and takes the maximum:

$$u_{\text{branch}(x)} = \max_{i \in \{1, \dots, n\}} u_i(x)$$

$$u_{\text{total}(x)} = u_{\text{outer}(x)} + u_{\text{branch}(x)}$$

This reduces the number of DUP nodes at the binding site to $u_{\text{total}(x)} - 1$ and composes naturally with nested branches. However, it still produces unnecessary ERAs: on branches that use fewer than max copies, the excess copies must be erased.

4.2.3. Split-Site DUP Placement

The optimal approach is to insert DUP nodes at **split sites**, points in the program where a value's ownership must diverge between independent continuations, rather than concentrating them at the binding site.

When a variable x is live across a branch point and also used outside that branch, the compiler emits one DUP before the branch: one output serves the outer uses and the other enters whichever branch executes. Each branch then handles its own internal DUPs independently.

Consider a concrete example:

```

let x = ... in
f x                -- 1 outer use

```

```

if cond1 then
  if cond2 then
    g x x x          -- 3 inner uses
  else
    h x              -- 1 inner use
else
  42                 -- 0 inner uses

```

Under split-site placement, this becomes:

```

let x = ... in
let (x_f, x_branch) = DUP(x) in
f x_f
if cond1 then
  if cond2 then
    let (x1, x2) = DUP(x_branch) in
    let (x11, x12) = DUP(x1) in
    g x11 x12 x2
  else
    h x_branch
else
  ERA(x_branch)
42

```

The per-path costs are:

Path	DUPs	ERAs	Uses of x
cond1 = T, cond2 = T	3	0	4
cond1 = T, cond2 = F	1	0	2
cond1 = F	1	1	1

On every path, the number of DUP nodes is exactly $u_{\text{path}(x)} - 1$ where $u_{\text{path}(x)}$ is the number of actual consumptions of x on that path. The only waste is a single DUP + ERA pair on the cond1 = F path, where one copy enters the branch but is immediately erased because that branch does not use x .

This waste is unavoidable without duplicating the outer code ($f\ x$) into each branch, a code-motion optimization that is semantically valid in a pure language but causes exponential code size growth with nesting depth. In practice, the single unnecessary ERA is negligible: for flat values ERA is a no-op and for heap values the DUP-ERA annihilation rule eliminates both the DUP and ERA in $O(1)$ with zero copies made.

We adopt split-site placement as Soma’s DUP placement strategy.

Definition. For a binding $\text{let } x = e \text{ in body}$, DUP nodes are inserted at points where x ’s ownership must diverge between independent continuations. The number of DUP nodes on any execution path for binding x is exactly $u_{\text{path}(x)} - 1$, where $u_{\text{path}(x)}$ is the number of consumptions of x on that path. On paths where x enters a branch but is not used, a single ERA node reclaims the unused copy.

Of course, this requires the compiler to be aware of the control flow structure of the program. Therefore, we can't use Lean 4's approach to pattern matching through higher-order functions in their core calculus which obscures the control flow graph.

4.2.4. Higher-Order and Recursive Usage

When a value is passed as an argument to a function, the caller transfers ownership. The callee receives exactly one owned copy. If the callee uses the argument multiple times, its own DUP chain handles the duplication, this is determined by the callee's own usage analysis, not the caller's.

For recursive functions, usage at each call site is always finite (one owned copy per argument per call). The recursion itself does not create unbounded static usage counts; each invocation independently manages its own arguments through DUP chains computed from the function body's usage analysis.

4.3. Ownership Rule

The duplication and erasure strategy is grounded in a single rule:

Every value has exactly one owner at every point in the program.

A DUP node takes one owned value and produces two independently owned values, the original ceases to exist as a distinct entity. An ERA node takes one owned value and destroys it. Every wire in the interaction net carries exactly one owned value.

This invariant is a direct consequence of the interaction net model. Lafont's interaction nets (1990) require that every port is connected to exactly one other port; this one-to-one wiring ensures unique ownership by construction.

The consequences are:

1. No reference counting is required for the common case.
2. No garbage collector is needed.
3. Deallocation is deterministic: every value is freed at a statically known program point (either at its last use, or at an ERA node on branches that do not use it).
4. Use-after-free is impossible: once a value is erased, no wire in the net references it.
5. Cyclic data structures cannot arise: the interaction net is a directed acyclic graph by construction, since every wire connects exactly two ports with no sharing.

4.4. Tiered Duplication Strategy

Not all values require the same duplication mechanism. The compiler classifies every type into one of two tiers at compile time and selects the appropriate DUP and ERA strategy accordingly.

4.4.1. Flat Tier: Register-Copyable Types

For integers, booleans, floats, characters, function pointers and structs composed entirely of flat fields, DUP is a register copy and ERA is a no-op (no heap allocation to free). Primitive values are represented as tagged words in the runtime: the low 3 bits encode the type tag and the remaining bits hold the payload. This has zero overhead, identical cost to Rust's Copy semantics.

The compiler uses a recursive predicate to determine whether a composite type is flat: a struct is flat if and only if all of its fields are flat. This means a `Point { x: Int, y: Int }` is duplicated with the same zero-overhead register copies as a bare integer.

For ERA, a symmetric predicate determines whether any sub-field contains a pointer. Flat types never do, so ERA on a flat value is always a no-op.

4.4.2. Heap Tier: Lazy Duplication via Superposition Nodes

All types that contain pointers such as closures, tagged unions, recursive data structures and any composite type with pointer-valued fields use lazy duplication via superposition nodes (SUPs).

When DUP is applied to a heap-tier value, instead of performing any copy, the runtime creates a SUP node wrapping the original value. A SUP represents a value that has been logically duplicated but whose copies have not yet been physically separated. Each consumer receives a projection reference (`proj0` or `proj1`) that resolves lazily: if only one projection is ever accessed, the original value is returned directly and no copy is made. Only when both projections are accessed does the runtime clone the value.

This design follows the symmetric interaction combinators of Lafont (1997), using the same agent types ($\gamma, \delta, \varepsilon$) and interaction rules.

Closures are a particularly important case. A closure is a heap-allocated object containing a function pointer, an arity and an array of captured environment values. When both projections of a duplicated closure are accessed, the runtime allocates a new closure header and copies the environment. For environment slots that contain pointers to other closures or SUPs, the cloner wraps them in fresh SUP nodes for lazy nested cloning. The deeply nested values are only actually copied if both copies are independently accessed. The runtime uses per-thread memory pools with size-class allocation (small closures ≤ 48 bytes, medium ≤ 112 bytes, large via `malloc`) to minimize allocation overhead.

For ERA, the compiler generates type-directed erasure code at compile time. Flat fields within a struct are skipped (no-op). Pointer fields are freed via the runtime. Tagged union payloads carry a count prefix that enables the runtime to walk and recursively free their fields without compile-time knowledge of the variant's layout. This approach avoids the need for compiler-generated drop functions while remaining sound for all type structures.

4.4.2.1. Labeling and Scope Management

Each DUP node in the compiled program carries a **label**, which is a natural number assigned statically at the DUP's source site. The label determines how DUP and SUP nodes interact. Matching labels trigger annihilation (the common case), while different labels trigger commutation (handling nested duplications).

Crucially, Soma does not employ an oracle. Lamping's original optimal reduction algorithm (1990) required an oracle in order to handle arbitrary untyped λ -terms. The oracle nodes accumulate during reduction and can cause exponential overhead, undermining the optimality claim in practice.

Soma avoids this problem entirely. In Soma's compiled output, every DUP node is emitted by the compiler at a known source location with a statically assigned label. The label assignment is deterministic: each syntactic DUP site receives a unique label at compile time. There is no dynamic label generation and no runtime scope tracking. When a DUP meets a SUP, the interaction is fully determined by comparing the two labels which is a single integer comparison, not a graph traversal.

The pathological self-copying terms that require the oracle are not expressible in Soma's type system. A well-typed term in Soma can duplicate values (via the DUP mechanism), but the structure of the duplication is always statically determined by the type checker's usage analysis. There is no mechanism by which a term can receive an opaque value and duplicate it in a way that the compiler did not anticipate.

4.4.2.2. Interaction Rules

The behavior of SUP nodes is governed by five interaction rules:

DUP-Constructor commutation. When a consumer accesses a SUP and finds a constructor underneath, the DUP pushes through the constructor: two new constructor cells are allocated (one per consumer) and each field is wrapped in a fresh SUP node rather than being copied. Duplication is deferred to the fields.

$$\text{DUP}^l(\text{Cons}(h, t)) \Rightarrow (\text{Cons}(\text{SUP}^l(h)_0, \text{SUP}^l(t)_0), \text{Cons}(\text{SUP}^l(h)_1, \text{SUP}^l(t)_1))$$

In simple terms, this is just the commutation rule of interaction combinators: the DUP and the constructor swap places, with the constructor duplicating the DUP's label and pushing it down to the fields.

DUP-ERA annihilation. When one consumer erases its copy, the DUP is eliminated entirely and the surviving consumer receives the original value. No copy is ever made. This is the key rule that makes defensive duplication (DUP at binding sites in the presence of branches) essentially free: if only one branch executes, the other branch's ERA annihilates the DUP.

SUP-ERA erasure. When an ERA meets a SUP node, the SUP is freed and both values inside it are recursively erased. This handles the case where a duplicated value is no longer needed by either consumer.

SUP-DUP same-label annihilation. When a DUP with label l encounters a SUP with the same label l , they cancel out. The two values inside the SUP are distributed directly to the two consumers. This is the common case: a value duplicated once, then both copies accessed costs $O(1)$.

$$\text{DUP}^l(\text{SUP}^l(a, b)) \Rightarrow (a, b)$$

SUP-DUP different-label commutation. When a DUP with label l_1 encounters a SUP with a different label l_2 , both push through each other, creating new SUP and DUP nodes with their respective labels preserved. This handles nested duplications correctly without oracle machinery.

$$\text{DUP}^{l_1}(\text{SUP}^{l_2}(a, b)) \Rightarrow (\text{SUP}^{l_2}(\text{DUP}^{l_1}(a)_0, \text{DUP}^{l_1}(b)_0), \text{SUP}^{l_2}(\text{DUP}^{l_1}(a)_1, \text{DUP}^{l_1}(b)_1))$$

4.4.2.3. Access-Site Code Generation

When compiled code accesses a value of recursive type (pattern match, field projection), the compiler emits a SUP check before the access.

This is one branch per access. The common case (the value is a real constructor, not a SUP) is highly predictable by modern branch predictors. After resolution, subsequent accesses hit the real value directly.

4.4.2.4. Asymptotic Advantage

Consider a list of N elements that is duplicated, where one consumer accesses only the first K elements:

Operation	Eager clone	Soma (lazy SUP)
Clone list, both use all N	$O(N)$	$O(N)$ (deferred)
Clone list, one uses $K < N$	$O(N)$	$O(K)$
Clone list, one side erased	$O(N)$	$O(1)$

The savings grow with the size of the data structure and the asymmetry of access patterns between consumers. In the common case where a value is duplicated defensively (because it appears in multiple branches) but only one branch executes, the DUP-ERA annihilation rule ensures zero copies are made.

4.4.2.5. Completeness of the Interaction Rules

We claim that the five rules above are complete for well-typed Soma programs. Every active pair (two agents whose principal ports are connected) has an applicable rule. The possible active pairs involving DUP, ERA, SUP and constructor agents are:

- $\text{DUP} \leftrightarrow \text{Constructor}$: handled by DUP-Constructor commutation.
- $\text{DUP} \leftrightarrow \text{ERA}$: handled by DUP-ERA annihilation.
- $\text{DUP} \leftrightarrow \text{SUP}$ (same label): handled by same-label annihilation.
- $\text{DUP} \leftrightarrow \text{SUP}$ (different label): handled by different-label commutation.
- $\text{ERA} \leftrightarrow \text{Constructor}$: standard interaction net erasure (ERA pushes through the constructor, erasing each field).
- $\text{ERA} \leftrightarrow \text{SUP}$: handled by SUP-ERA erasure.
- $\text{Constructor} \leftrightarrow \text{Constructor}$ (same type): standard annihilation.
- $\text{SUP} \leftrightarrow \text{SUP}$: cannot form an active pair. A SUP's principal port connects to a consumer (a DUP, ERA, or a pattern match), not to another SUP.

No other active pairs arise in well-typed programs. In particular, $\text{DUP} \leftrightarrow \text{DUP}$ does not occur because DUP nodes are always separated by the values they are duplicating.

4.5. Multithreading Considerations

Soma's runtime employs a work-stealing scheduler with per-thread Chase-Lev dequeues. When a duplicated value is consumed by different threads across a fork/join boundary, the SUP node becomes shared memory. Resolution of a SUP by two threads simultaneously would constitute a data race.

Soma adopts a thread-local SUP policy: when forking a task that captures one side of a duplicated value, the value is eagerly deep-copied at the fork boundary. SUP nodes exist only within a single thread’s subgraph. This avoids all contention since each worker operates on independently owned data.

This policy is simple and predictable. It can be refined in the future if profiling reveals that cross-thread duplication of large recursive structures is a bottleneck. Alternative strategies include atomic CAS-based SUP resolution (adding one atomic operation per cross-thread SUP resolution) and ownership transfer at fork boundaries (the parent gives up its side of the SUP to the child, ensuring each thread owns exactly one side with no sharing).

4.6. Comparison with HVM and Bend

Several systems share design goals with Soma. We compare along four axes: type system, execution model, parallelism and memory management.

4.6.1. Interaction Net Evaluators: HVM and Vine

HVM [4] and Vine [10] are interaction net interpreters. Both maintain the net as a runtime data structure and execute by graph rewriting: scanning for active pairs and applying rules via tag dispatch. This gives them automatic parallelism (strong confluence) and, in HVM’s case, optimal sharing (Lévy-optimality), but every value, even integers and booleans, pays the cost of graph node representation, pointer chasing and dispatch overhead.

Soma differs in three respects. First, it *compiles* interaction rules to native code rather than interpreting them. Statically predictable interactions (beta reduction, pattern matching, arithmetic) emit the same LLVM IR a conventional compiler would produce) and only dynamic interactions (DUP commuting through unknown values) require runtime infrastructure. Second, QTT’s quantity annotations eliminate interaction net overhead entirely for quantity 0 (erased) and quantity 1 (linear) bindings, which constitute the majority of bindings in typical programs. Third, Soma uses flat native data representations (tagged words, packed structs, array-backed lists) rather than graph nodes, yielding cache-friendly memory access patterns.

Soma is not Lévy-optimal. When both copies of a duplicated closure are accessed, they are cloned and reduced independently. This is a deliberate trade-off: Lévy-optimality minimizes β -reduction steps, but each step in an interpreter involves pointer indirection and cache misses. For well-typed programs, the pathological cases requiring optimal sharing (self-application of untyped terms) cannot arise and QTT further constrains duplication to ω -quantity bindings only.

However, Soma preserves the interaction net property that HVM and Vine exploit for fusion. Tier 2 values maintain graph topology at runtime and DUP-SUP annihilation compiles to native code. Self-inverse compositions (such as $\text{not} \circ \text{not} = \text{id}$) still fuse in $O(1)$ per annihilation, preserving the $O(\log N)$ reduction count for N -fold self-composition.

4.6.2. Native Compiled Languages: Lean and Koka

Lean [8] compiles to C with reference counting and copy-on-write (RC+COW). Every function boundary requires increment and decrement operations and when a reference count

exceeds 1, mutation triggers a full copy. Lean’s data structures are pointer-heavy linked lists, sacrificing cache locality.

Koka [9] uses Perceus reference counting with reuse analysis, achieving deterministic memory management with good single-thread performance. Like Lean, it lacks dependent types and automatic parallelism.

Soma shares the native compilation target but replaces reference counting with interaction net semantics (DUP/ERA/SUP). This has several consequences:

- No increment/decrement overhead on the hot path. Quantity 1 bindings have zero ownership-tracking cost.
- Lazy duplication via SUP defers cloning to the point of divergence. RC+COW pessimistically maintains counts everywhere and copies the moment any consumer mutates.
- DUP-ERA annihilation eliminates unnecessary copies in $O(1)$. RC requires decrement-and-conditional-free on every scope exit.
- No cycle problem. Interaction nets are acyclic by construction. RC requires cycle detection or backup tracing for cyclic data.

The trade-off is that structural sharing is less natural than RC+COW. When two consumers independently mutate a shared structure, RC+COW copies on first mutation; Soma’s DUP creates independent copies eagerly (Tier 1) or lazily (Tier 2), but does not support in-place mutation of shared data.

4.6.3. Quantitative Type Theory: Idris 2

Idris 2 [2] implements QTT with the same quantity semiring $(0, 1, \omega)$. However, Idris 2 uses QTT primarily as a correctness mechanism and targets high-level backends (Scheme, JavaScript, RefC). It does not exploit quantity annotations for compilation tier selection or memory management optimization.

Soma uses QTT as both a correctness mechanism and a *compilation strategy oracle*. The quantity of every binding directly determines its compilation tier ($0 \rightarrow$ erased, $1 \rightarrow$ native LLVM, $\omega \rightarrow$ compiled inet rules), its memory management strategy (no-op, deterministic free, lazy SUP) and its parallelism eligibility (ω DUP sites are candidate fork points).

4.6.4. Summary

Property	HVM	Vine	Lean	Soma
Type system	Untyped	Generics	Dependent	Dependent + QTT
Execution	Interpreted inet	Interpreted inet	Native (C)	Compiled inet + LLVM
Flat types	Graph nodes	Graph nodes	Boxed / unboxed	Register copy (zero cost)
Duplication	Optimal sharing	Lazy SUP	RC + COW	Lazy SUP + compiled rules

Property	HVM	Vine	Lean	Soma
Fusion	Yes	Yes	No	Yes (Tier 2)
Parallelism	Auto (GPU+CPU)	Auto (CPU)	None	QTT-guided (CPU)
Cache behavior	Poor	Poor	Moderate	Good (flat arrays)
GC-free	Yes	Yes	Yes (RC)	Yes (inet)

5. Compilation Pipeline

Soma compiles source code through a sequence of intermediate representations, each serving a distinct purpose. The pipeline is:

Source $\rightarrow$ CST $\rightarrow$ AST $\rightarrow$ Core $\rightarrow$ Circuit IR $\rightarrow$ Alloy IR $\rightarrow$ LLVM IR $\rightarrow$ Native

5.1. Frontend: Source to Core

The lexer produces tokens which the parser assembles into a Concrete Syntax Tree (CST) using a lossless green/red tree representation that preserves whitespace and comments for tooling. The CST is lowered to an Abstract Syntax Tree (AST), which is then elaborated into Core expressions.

Elaboration performs bidirectional type inference, unification (with row polymorphism), QTT usage checking, instance resolution for type classes and totality checking. The output is a fully annotated Core expression where every binding carries its quantity (0, 1, or ω) and every subexpression has a known type. Normalization uses evaluation by normalization (NbE) with defunctionalized closures.

5.2. Circuit IR: Interaction Net Graph

The Core expression is lowered to Circuit IR, Soma’s interaction net representation. Circuit IR encodes programs as graphs of nodes connected by ports, using a 64-bit term encoding:

SUB(1) | TAG(7) | EXT(24) | VAL(32)

The 7-bit tag field accommodates 20 node types: variable, lambda, application, duplication, superposition, erasure, constructor, match, record, projection, number, unary and binary operators, reference, use, ALO (lazy allocation), array, index, string and slice.

Each definition in the program becomes an entry in the Circuit IR *book* which is a static array of definition bodies represented as subgraphs. References between definitions use REF and ALO nodes:

- **REF** nodes reference definitions that accept arguments (higher-order). When forced, the entire definition body is instantiated via subgraph copy, but internal REF and ALO nodes within the copy are preserved as-is, providing call-granularity lazy instantiation.
- **ALO** (lazy allocation) nodes reference self-recursive or nullary definitions. They behave identically to REF during reduction but signal to the compiler that the definition is potentially recursive and should be instantiated lazily.

This lazy instantiation strategy ensures that recursive definitions unfold one call at a time rather than eagerly expanding to infinite depth.

5.3. Alloy IR: Monomorphized SSA

Circuit IR is partially evaluated (interaction rules are applied at compile time to simplify the graph) and then lowered to Alloy IR, a monomorphized SSA-form IR. Alloy uses conventional SSA concepts like local IDs, block IDs, function IDs, phi nodes augmented with interaction net primitives:

- `lazySup label value typeDesc` create a SUP node (Tier 2 lazy duplication)
- `supProj0 sup / supProj1 sup` project from a SUP
- `clone value type-specialized` eager copy (Tier 1 duplication)
- `erase value type-specialized` destruction

Types in Alloy include primitives, pointers, structs, tagged unions, closures (function pointer + environment) and arrays. Every type is fully monomorphized and no polymorphism remains.

5.4. LLVM Code Generation

Alloy IR is lowered to LLVM IR, each Alloy function becomes an LLVM function and each block becomes an LLVM basic block. The interaction net primitives lower to calls into the C runtime (`soma_runtime.c`) or to inline LLVM operations:

- Flat-tier DUP → register copy (zero cost)
- Flat-tier ERA → no-op
- Heap-tier eager clone → call to type-specialized `clone_fn` from `SomaTypeDesc`
- Heap-tier lazy SUP → call to `soma_dup_typed`
- SUP projection → call to `soma_proj0 / soma_proj1`
- Erasure → call to type-specialized `erase_fn` from `SomaTypeDesc`

The `SomaTypeDesc` structure bundles a clone function pointer and an erase function pointer, both generated at compile time as static LLVM globals. Each concrete type gets exactly one `SomaTypeDesc`, enabling SUP nodes to carry a single pointer (8 bytes) rather than two function pointers (16 bytes).

6. Compiled Interaction Net Reduction

The central contribution of Soma's design is the compilation of interaction net reduction rules to native code. Rather than maintaining the interaction net as a runtime data structure (as in HVM or Vine) or compiling it away entirely (as in a conventional compiler), Soma compiles the reduction *itself* to type-specialized native code. This section formalizes the approach.

6.1. Statically Predictable Interactions

The key observation is that the vast majority of interactions in a well-typed program are statically predictable. At each point in the compiled program, the compiler knows which interaction rule will fire:

- **APP-LAM** (beta reduction): The compiler knows a function is being applied. This compiles to a direct `call` instruction.

- **MAT-CTR** (pattern match): The compiler knows a scrutinee is being matched. This compiles to a switch on the constructor tag.
- **PROJ-RECORD** (field access): The compiler knows a field is being projected. This compiles to a load at a known offset.
- **OP-NUM** (arithmetic): The compiler knows operands are numbers. This compiles to native arithmetic instructions.

These interactions require no interaction net infrastructure at runtime. They compile to exactly the same LLVM IR that a conventional functional language compiler would produce.

The only interactions that require runtime infrastructure are those involving *duplication and sharing*:

- **DUP-LAM** commutation: Duplicating a closure creates two copies.
- **DUP-SUP** annihilation: A duplication meeting its own superposition cancels out.
- **DUP-SUP** commutation: Duplications at different nesting levels pass through each other.
- **DUP-ERA** annihilation: An unnecessary duplication is eliminated.

These are the dynamic interactions and they only arise for ω -quantity bindings.

6.2. Three-Tier Compilation

QTT's quantity annotations partition every binding into one of three compilation tiers:

Tier 0: Static erasure (quantity 0): The binding exists only for type checking. No runtime representation or code is generated. This handles proofs, type-level computations and compile-time indices. The cost is zero and strictly better than any system that represents erased terms at runtime.

Tier 1: Native code (quantity 1 and quantity ω with eager clone): The binding is consumed at most once (quantity 1) or is duplicated but both copies are consumed immediately (quantity ω with small types). The compiler emits conventional LLVM IR:

- Function application $\rightarrow$ call
- Pattern matching $\rightarrow$ switch
- Field access $\rightarrow$ getelementptr + load
- Duplication $\rightarrow$ type-specialized clone_fn (memcpy for flat data, deep copy for pointers)
- Erasure $\rightarrow$ type-specialized erase_fn

No SUP nodes, no interaction net graph, no lazy sharing. This is the same code Lean, Koka, or any conventional compiled language would produce. For programs that are predominantly linear (most programs), the entire program runs at this tier.

Tier 2: Compiled interaction net rules (quantity ω with lazy sharing). For bindings where two consumers may access a value at different times (one immediately, one much later, or one conditionally), the compiler emits lazy duplication via SUP nodes with compiled reduction rules:

- DUP creates a SUP node wrapping the value (call to soma_dup_typed)
- Each consumer receives a projection (soma_proj0 / soma_proj1)
- If only one projection is accessed, the original value is returned directly (zero copy)

- If both are accessed, the type-specialized `clone_fn` is invoked (same as Tier 1)
- DUP-SUP same-label annihilation: $O(1)$ with only two stores and zero allocation
- DUP-SUP different-label commutation: type-specialized native function that creates the crossed SUP/DUP structure

The critical property of Tier 2 is that it *preserves graph topology*. SUP nodes, DUP nodes and the wires connecting them exist at runtime. This is what enables the two properties that compiling away the interaction net would sacrifice: runtime fusion and parallel reduction.

6.3. Runtime Fusion

When two copies of a self-inverse function are composed via sharing (SUP), the intermediate forms can cancel out through DUP-SUP annihilation. This yields an exponential reduction in interaction count for certain patterns.

Consider the Church-encoded boolean negation:

$$\text{not} = \lambda b. \lambda t. \lambda f. (b \ f \ t)$$

Applying `not` to itself yields the identity: $\text{not} \circ \text{not} = \text{id}$. In an interaction net, this cancellation happens via DUP-SUP annihilation when the two copies of `not` share their input through a SUP node. Composing `not` 2^K times requires only $O(K)$ interactions rather than $O(2^K)$ function applications.

This property is *structural*: it depends on the graph topology (SUP nodes connecting shared subexpressions) and the annihilation rule (same-label DUP-SUP cancels in $O(1)$). Both are preserved in Tier 2's compiled representation. The annihilation rule compiles to a single label comparison followed by two stores native-speed execution of the same rule that HVM interprets.

Tier 2 values at quantity ω are exactly the values where fusion matters. Quantity 0 values don't exist at runtime. Quantity 1 values are never duplicated, so fusion is irrelevant. QTT routes values to the tier where their optimization properties are maximally exploited.

6.4. Parallelism via Compiled Reduction

Interaction nets have a structural property that conventional computation models lack: **strong confluence**. The number of reduction steps required to reach normal form is independent of the order in which rules are applied. Any two non-interfering active pairs can reduce simultaneously without coordination and the result is guaranteed to be the same.

In compiled code, this translates directly: if two compiled reduction rules operate on disjoint subgraphs, they can execute on different threads. The only change required is replacing the LINK operation (a store to a port) with a CAS (atomic compare-and-swap). The reduction rules themselves are identical.

For all **Quantity ω DUP sites**, each DUP creates two independent subgraphs. These are the natural fork points for parallel execution.

The compiler can insert fork/join directives at DUP sites for ω -quantity bindings at compile time, without runtime analysis. Per-thread arena allocation (extending the existing pool

allocator) eliminates contention on the allocation fast path. A work-stealing scheduler with Chase-Lev dequeues [3] distributes work across cores.

For IO, the linear IO token (quantity 1) prevents duplication statically: `IO.fork` is the only mechanism to split the token, providing explicit controlled parallelism. This is enforced by the type system at compile time, not by runtime checks.

7. Conclusion and Future Work

We have presented Soma, a dependently typed functional language that compiles interaction net reduction rules to native code via LLVM. The design exploits Quantitative Type Theory as a compilation strategy oracle and a three-tier architecture to occupy a previously unexplored point in the design space: native single-thread performance for the linear case with interaction net benefits for the unrestricted case.

The memory management strategy derives DUP and ERA primitives from interaction net semantics, achieving GC-free deterministic lifetimes without reference counting. Split-site DUP placement minimizes duplication to the exact points where ownership diverges and the tiered duplication strategy (register copy for flat types, lazy SUP for heap types) ensures that no unnecessary copies are made. DUP-ERA annihilation makes defensive duplication at branch points essentially free.

Bibliography

- [1] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2018. 56–65.
- [2] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP)*, 2021.
- [3] David Chase and Yosef Lev. 2005. Dynamic Circular Work-Stealing Deque. In *Proceedings of the 17th Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2005. 21–28.
- [4] HigherOrderCo. 2024. HVM: A Massively Parallel Optimal Functional Runtime. Retrieved from <https://github.com/HigherOrderCO/HVM>
- [5] Yves Lafont. 1990. Interaction Nets. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1990. 95–108.
- [6] Yves Lafont. 1997. Interaction Combinators. *Information and Computation* 137, 1 (1997), 69–101.
- [7] Conor McBride. 2016. I Got Plenty o' Nuttin'. *A List of Successes That Can Change the World*, 207–233.
- [8] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction -- CADE 28*, 2021. Springer, 625–635.

- [9] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. 2021. Perceus: Garbage Free Reference Counting with Reuse. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2021. 96–111.
- [10] Vine Contributors. 2024. Vine: A Functional Programming Language with Interaction Net Evaluation. Retrieved from <https://github.com/vine-lang/vine>